

Architectural Risk Assessment: SQS FIFO Pipeline Redesign

[Proposal Evaluation]: Splitting Combined ARM Events via MessageGroupId = Env_ID

[Status]: Critical Severity Risk — Not Recommended

1. Executive Summary

Attempting to split multi-environment ARM event payloads into single-environment messages to process them in parallel using 'currentDeployEnvId' as the SQS MessageGroupId introduces critical runtime bugs, data state synchronization flaws, and upstream contract violations.

This audit evaluates the foundational data models and core API clients of the ACIS framework. The splitting proposal directly violates the framework's state-accumulation contract and destroys connection pooling mechanisms, rendering core reporting and tracking functionalities unfixable.

2. Stateful Code-Level Failure Analysis (New Findings)

Critical Risk #1: Memory Isolation & Context Accumulation Fragmentation

Why it fails: The ACIS framework separates data into immutable event inputs and a mutable, shared memory scratchpad ('ContextData'). The state machine is designed to run sequentially across environments to accumulate execution results into arrays before executing the final notification steps. Splitting events completely shatters this shared memory model across isolated Lambda runtimes.

Exact Code Location That Breaks (In 'states/base_state.py'):

```
@dataclassclass ContextData(BaseContextData):...email_context_list: list =\n    field(default_factory=list)notification_results: Dict[str, Any] =\n    field(default_factory=dict)site_id_context_list: list = field(default_factory=list)
```

The Catastrophe: In the existing architecture, as the single Lambda loops through Dev, UAT, and Prod, it appends tracking tokens, site metadata, and results into 'email_context_list' and 'notification_results'. At the conclusion of all environments, 'PostArmEventNotificationState' processes this compiled state data to send a single consolidated execution report back to Siemens. If you split the payload, separate Lambda instances initialize completely blank, isolated 'ContextData' objects. Lambda 1 only knows about Dev; Lambda 2 only knows about UAT. It becomes architecturally impossible to compile a unified summary, leaving downstream state layers completely blind to the overall transaction lifecycle.

Critical Risk #2: ID Duplication & Metric Correlation Collision

Why it fails: Tracking keys and telemetry signatures are generated by parsing the immutable request payload. Splitting creates an absolute data-contract conflict where separate, uncoordinated server processes run on identical transaction keys.

Exact Code Location That Breaks (In 'states/base_state.py'):

```
@classmethoddef from_dict(cls, data: Dict[str, Any]) -> "UserProvisioningEventData":\n    return cls(... correlation_id=data["requestId"],
```

The Catastrophe: The engine instantiates 'UserProvisioningRequestHandler' and maps core tracing to 'correlation_id'. If messages are split, multiple parallel Lambdas will execute with the exact same 'requestId'.

When separate runtime handlers concurrently attempt to register logs, update transaction metrics, or check operation idempotency tables using 'correlation_id' as a unique query parameter, they will trigger race conditions, resulting in duplicate telemetry records, data pollution, or database index crashes.

Severity 2 Risk #3: API Socket Exhaustion & Destination Target Flooding

Why it fails: The framework utilizes an explicit connection-pooling model to optimize network efficiency and maintain a low, stable socket footprint when communicating with HashiCorp Vault and Teamcenter.

Exact Code Location That Breaks (In 'ApiClient' Class):

```
class ApiClient: def __init__(self, headers: Optional[Dict[str, str]] = None): ...
self._session = requests.Session() self._session.headers.update(self.headers)
```

The Catastrophe: 'requests.Session()' establishes a persistent connection pool, reusing open TCP sockets sequentially across the environment processing sequence.

If payloads are split across separate Lambda instances, every single environment spin-up instantiates an independent 'ApiClient' pool. Instead of reusing open sockets safely, the architecture floods Vault and Teamcenter endpoints with concurrent, overlapping TCP connection handshakes. This triggers immediate rate-limiting or socket exhaustion errors at the destination API gateways, causing intermittent onboarding drops.

Critical Risk #4: Asymmetric State Evaluation & Rogue User Deactivations

Code: `previous_deploy_env_ids = list(set(previous_ids) - set(current_ids))`

Breach: Splitting blinds individual environment contexts. Dev Lambda calculates set difference without seeing UAT is active, mistakenly triggering 'UpsTcxUserDeactivationState' and deleting active users from companion environments.

Critical Risk #5: Multi-Tenant Concurrency Starvation (Noisy Neighbor)

System Limit: Max Lambda Concurrency Capacity = 5 Threads

Breach: If Customer A triggers an update spanning 5 environments, splitting spins up 5 parallel Lambdas, consuming 100% of the entire application compute pool. Customer B's incoming transactions are completely starved and blocked in the queue.

4. The Winning Solution: Inter-Tenant Batching Architecture

Instead of altering payloads or forcing high-risk code refactors, optimize the pipeline safely at the AWS infrastructure tier:

- Set SQS MessageGroupId = ECA_ID (Tenant ID)
- Set SQS Lambda Source Batch Size = 5
- Enable 'Report Batch Item Failures' in Lambda Trigger Settings

Why This Is Vastly Superior and Compliant:

- **Preserves Context Accumulation:** Payloads remain unified. Data lists like 'email_context_list' gather state sequentially inside a single execution thread, allowing notifications to compute flawlessly out-of-the-box.
- **Fair Compute Distribution:** Bounds each customer tenant to exactly 1 active Lambda slot. Customer A cannot monopolize the 5-thread capacity, keeping 4 slots open for Customer B, C, D, and E to process in parallel.
- **Network-Safe Session Re-use:** Maximizes the efficiency of the 'requests.Session()' connection pool inside 'ApiClient', maintaining a tight, clean socket profile.
- **Zero Refactoring Overhead:** Zero structural changes required for business state logic. We only wrap 'lambda_handler' in a clean records iteration loop to support partial batch returns ('batchItemFailures').